

第二章

并发进程

主要内容

- 2.1 进程及其调度控制与死锁**
- 2.2 UNIX的进程**
- 2.3 中断的基本概念及UNIX中断处理**
- 2.4 UNIX时钟中断与系统调用**
- 2.5 UNIX的进程调度状态**
- 2.6 进程通讯**

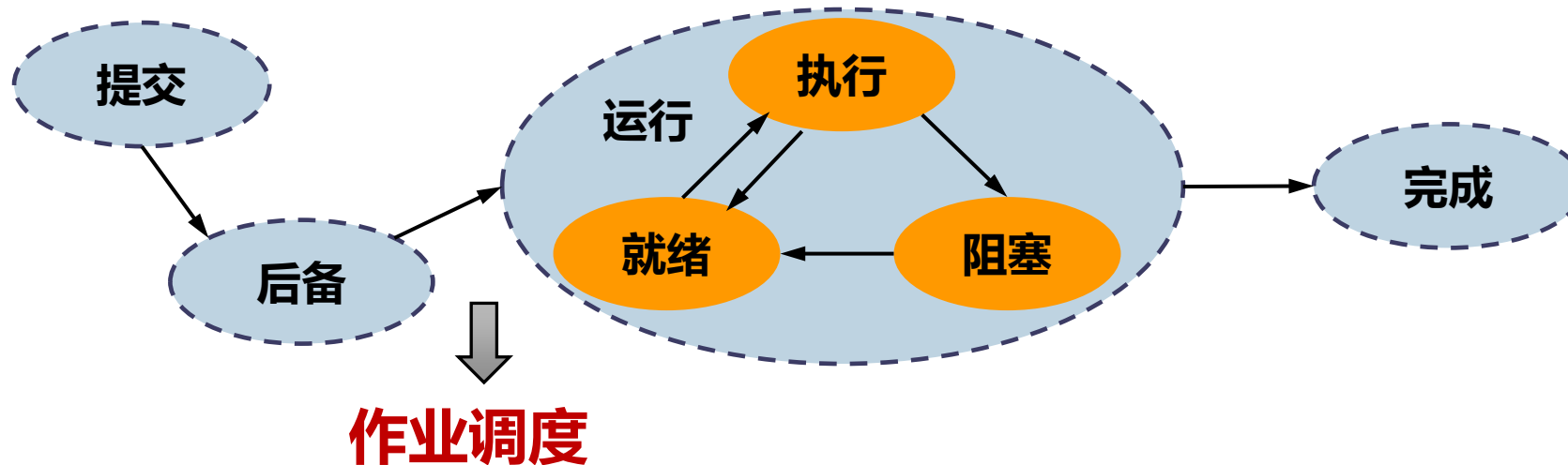


处理机调度



处理机调度

采用多道批处理技术后，作业从进入到退出系统大致经历四个阶段：



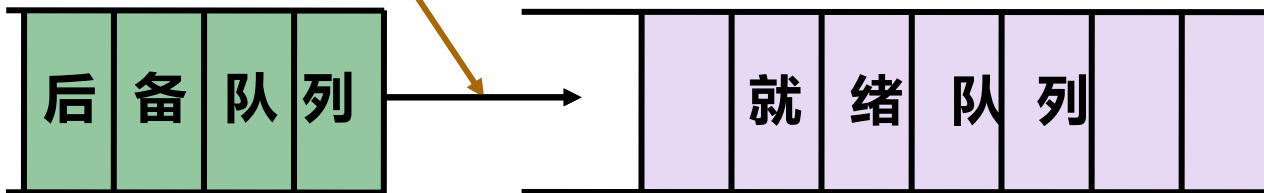
- 接纳多少个作业：取决于系统的**多道程序度**（系统规模和运行速度）
- 接纳哪些作业：取决于具体的**调度算法**

作业调度过程：

1. 按照一定的策略选取若干作业调入内存
2. 创建进程，分配资源
3. 如创建成功，将新创建的进程插入就绪队列



作业调度



具有一级调度的调度队列模型

- 外存
- 内存
- 交换区

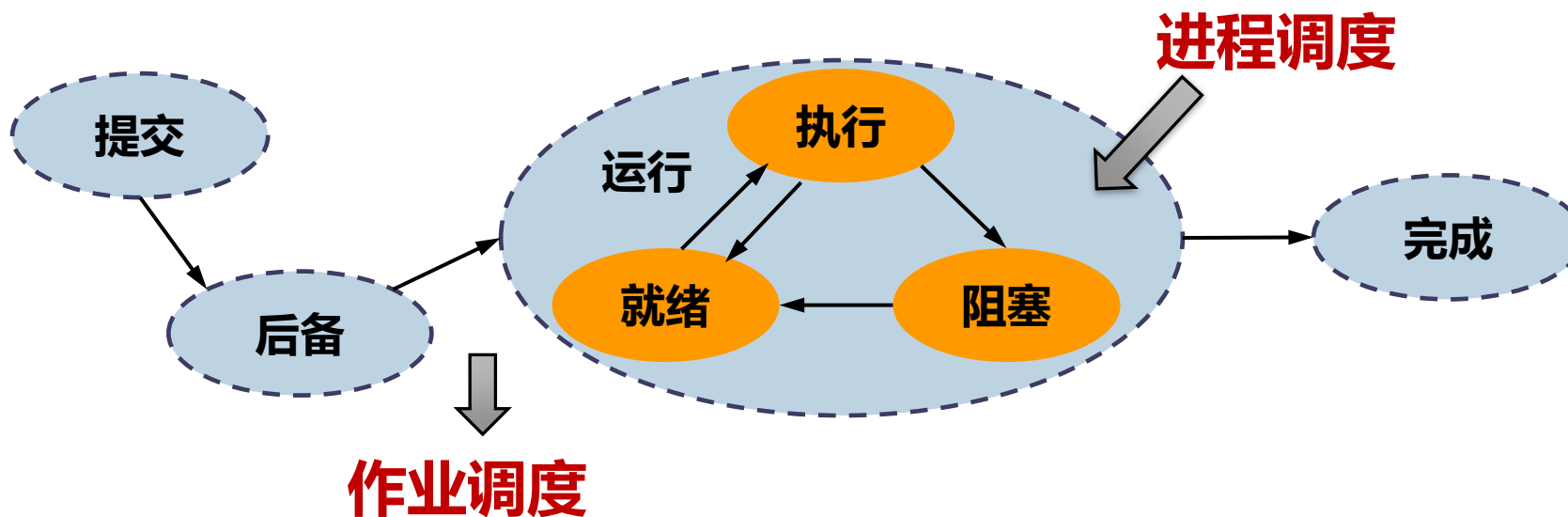


处理机调度



处理机调度

采用多道批处理技术后，作业从进入到退出系统大致经历四个阶段：



- 接纳多少个作业：取决于系统的**多道程序度**（系统规模和运行速度）
- 接纳哪些作业：取决于具体的**调度算法**

作业调度过程：

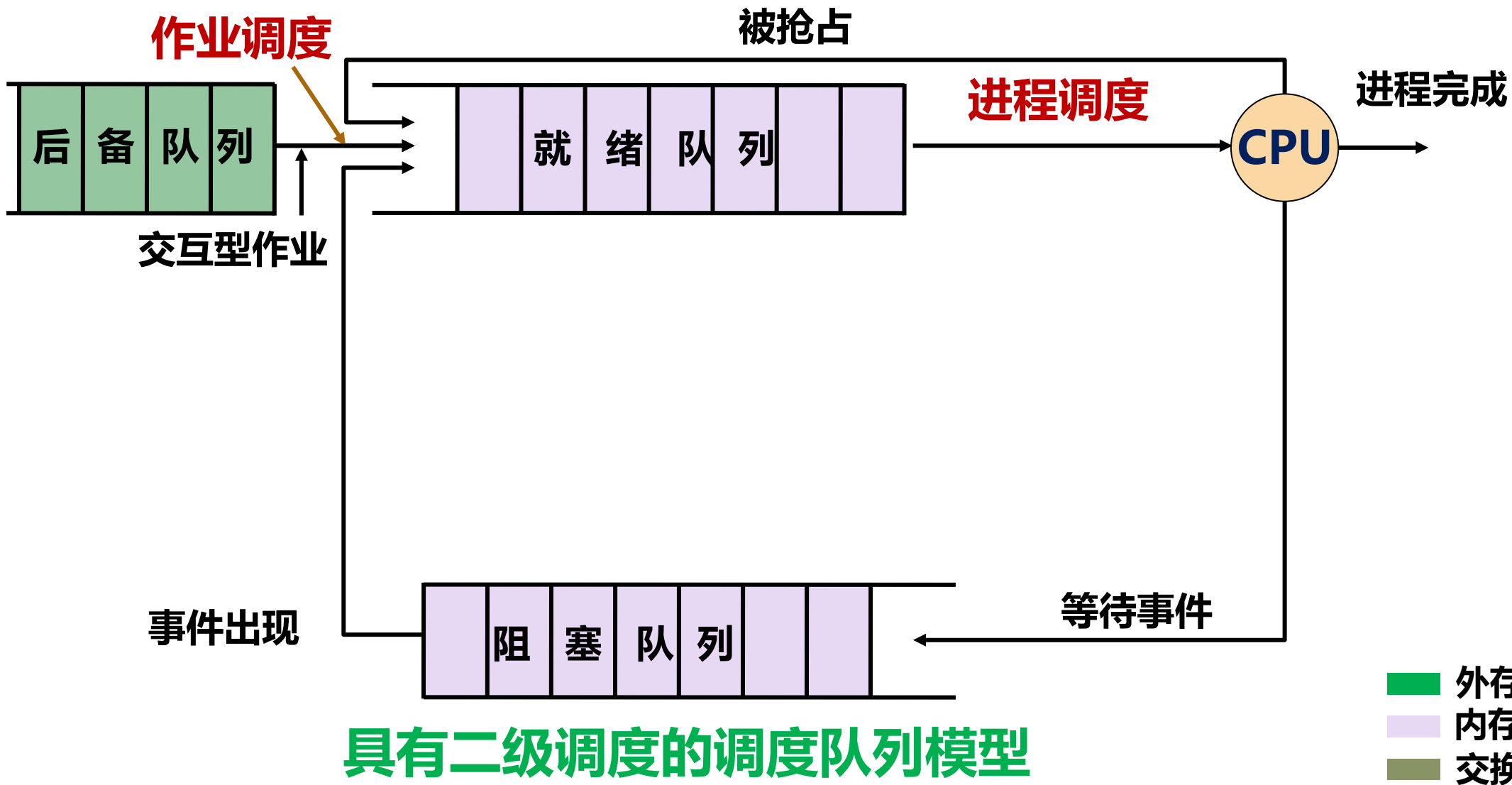
1. 按照一定的策略选取若干作业调入内存
2. 创建进程，分配资源
3. 如创建成功，将新创建的进程插入就绪队列



处理机调度



处理机调度

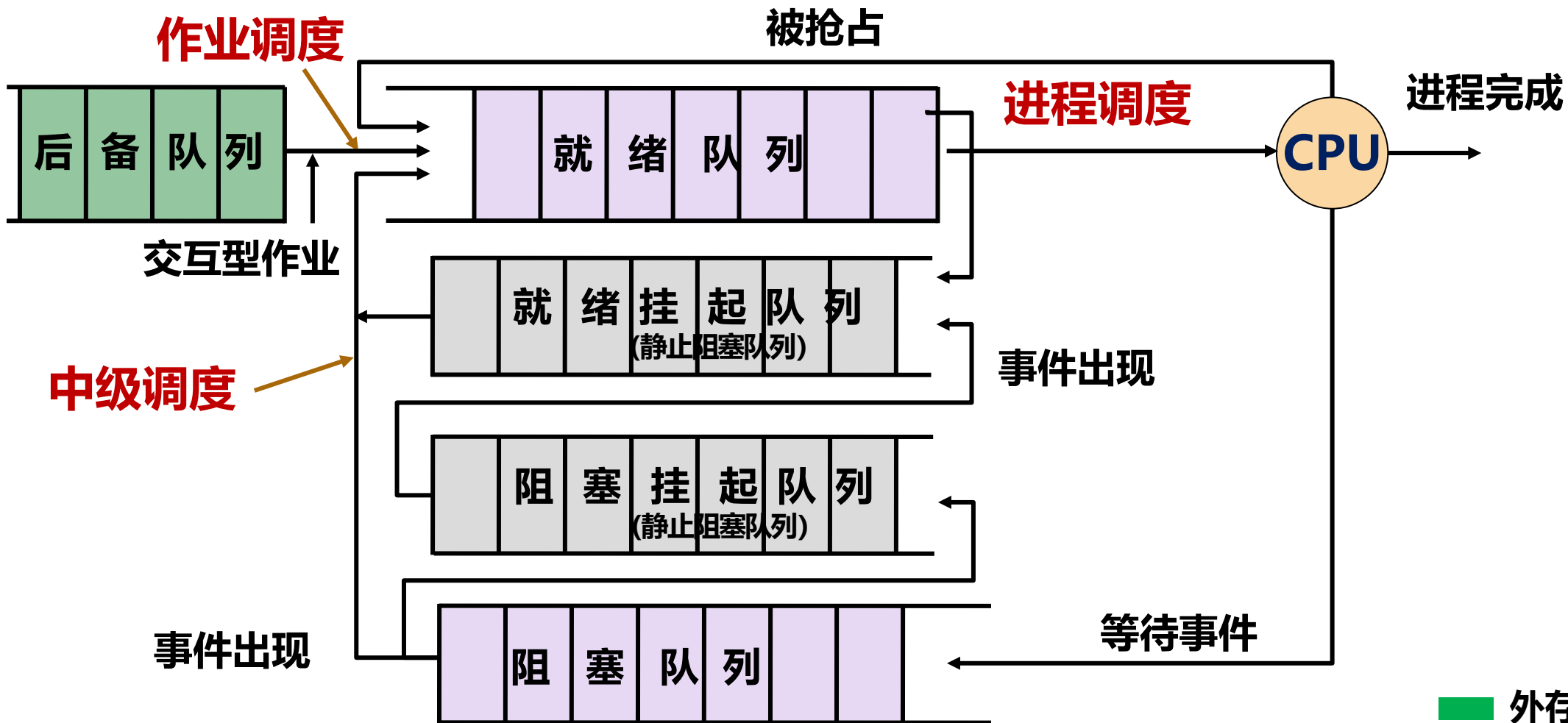




处理机调度



处理机调度



具有三级调度的调度队列模型

- 外存
- 内存
- 交换区



1. 调度本身的开销
2. 相关数据结构的维护
3. 各种不同的调度对象：I/O密集型，CPU密集型，长作业，短作业



选择调度方式和调度算法的若干准则

1. 周转时间短——批处理型作业

$$\begin{aligned}\text{周转时间} &= \text{作业完成时间} - \text{作业提交时间} \\ &= \text{作业运行时间} + \text{作业等待（后备）时间}\end{aligned}$$

2. 响应速度快——交互型作业
3. 高优先权优先
4. 截至时间保证——实时型作业



处理机调度

1. 先来先服务算法 (FIFO)

作业调度

从后备队列中选择一个或多个最先进入该队列的作业，将它们调入内存、为它们分配资源、创建进程，放入就绪队列。

优点：

非抢占式，实现简单。

缺点：

1. 长作业（进程）有利，短作业（进程）吃亏；
2. CPU密集型作业（进程）有利，I/O密集型作业（进程）吃亏。

进程调度

从就绪队列中选择一个最先进入该队列的进程，为之分配处理机。该进程一直运行到完成或阻塞时才放弃处理机。

非抢占

周转时间
响应时间
优先权





处理机调度

2. 短作业/进程优先 (SJ/PF)

作业调度

从后备队列中选择一个或多个估计运行时间最短的作业，将它们调入内存、为它们分配资源、创建进程，放入就绪队列。

进程调度

从就绪队列中选择一个估计运行时间最短的进程，为之分配处理机。该进程一直运行到完成或阻塞时才放弃处理机。

非抢占

周转时间
响应时间
优先权



优点:

当所有作业/进程同时到达时，平均周转时间最短

缺点:

1. 短作业（进程）有利，长作业（进程）吃亏；
2. 未考虑作业（进程）的紧迫程度。



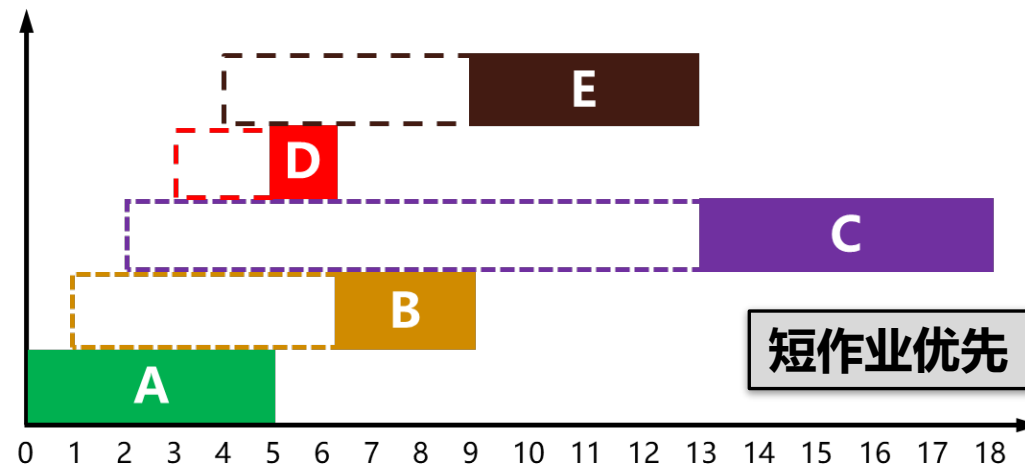
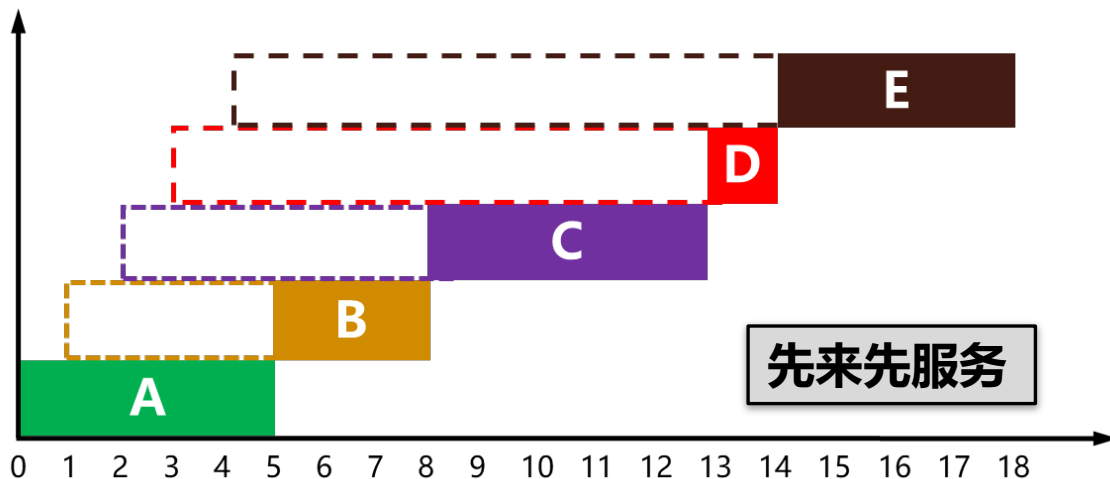
处理机调度



周转时间
响应时间
优先级

进程调度算法周转时间比较

进程名	A	B	C	D	E	平均
到达时间	0	1	2	3	4	
服务时间	5	3	5	1	4	
周转时间 (先来先服务)	5	7	11	11	14	9.6
周转时间 (短作业优先)	5	8	16	<u>3</u>	9	8.2





处理机调度

2. 短作业/进程优先 (SJ/PF)

作业调度

从后备队列中选择一个或多个估计运行时间最短的作业，将它们调入内存、为它们分配资源、创建进程，放入就绪队列。

优点：

当所有作业/进程同时到达时，平均周转时间最短

缺点：

1. 短作业（进程）有利，长作业（进程）吃亏；
2. 未考虑作业（进程）的紧迫程度。

进程调度

从就绪队列中选择一个估计运行时间最短的进程，为之分配处理机。该进程一直运行到完成或阻塞时才放弃处理机。

非抢占

抢占式改进：
最短剩余时间优先

周转时间
响应时间
优先权





处理机调度

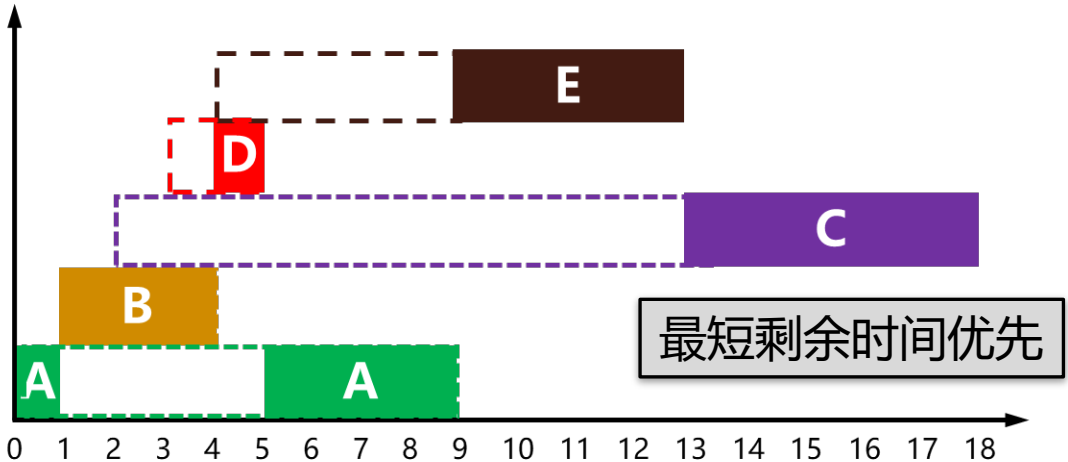
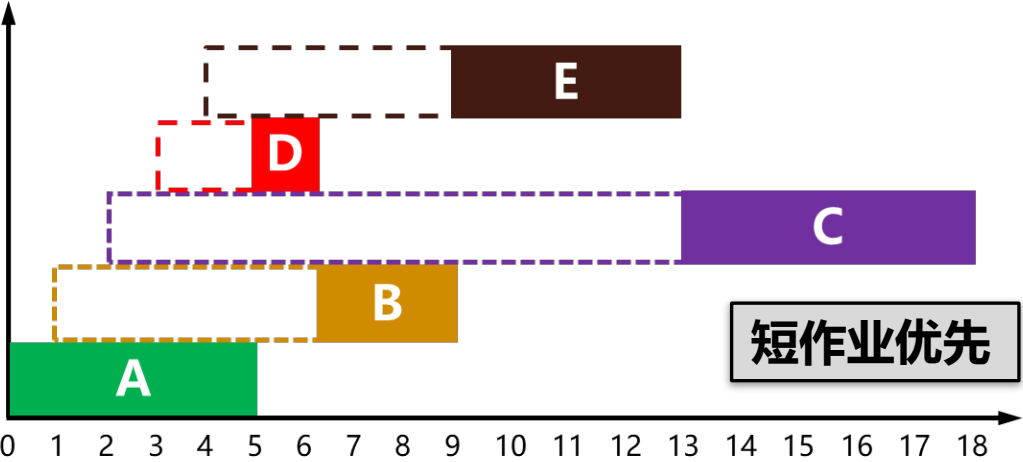


周转时间
响应时间
优先权

进程调度算法周转时间比较

进程名	A	B	C	D	E	平均
到达时间	0	1	2	3	4	
服务时间	5	3	5	1	4	
周转时间 (先来先服务)	5	7	11	11	14	9.6
周转时间 (短作业优先)	5	8	16	<u>3</u>	9	8.2
周转时间 (最短剩余时间优先)	9	3	16	<u>2</u>	9	7.8

调度算法





处理机调度



周转时间
响应时间
优先权



3. 时间片轮转调度算法 (RR)

交互式系统 (分时系统)
中的进程调度



时间片大小将
影响系统性能

进程调度

每个进程轮流使用CPU固定时间片后将CPU让给其它进程，自己进入就绪队列等待下一轮调度。

抢占



优点：各进程能够比较均衡地共享使用处理机。

缺点：系统的效率与时间片的设置密切相关。时间片过大，与用户的交互性就差；时间片过小，进程间切换过于频繁，一个进程需要轮转多次才能到达终点，系统开销就会增大。



处理机调度

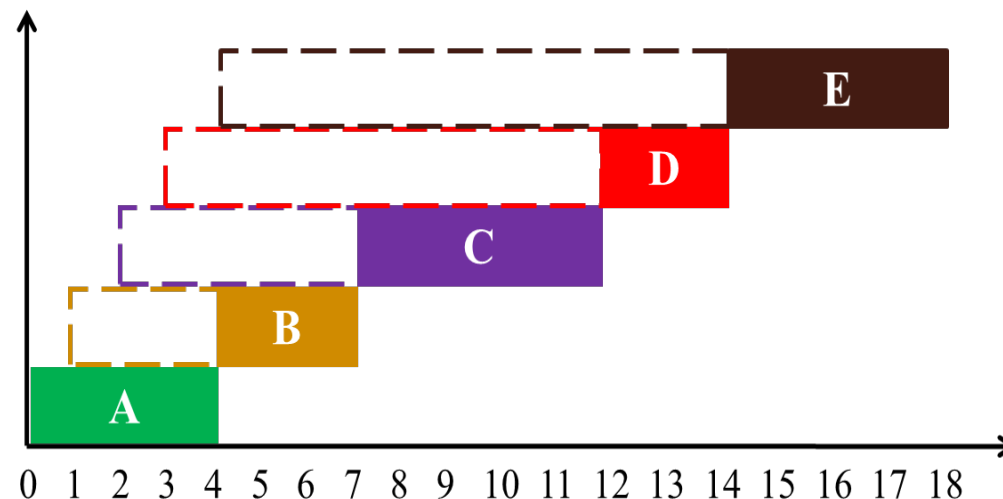
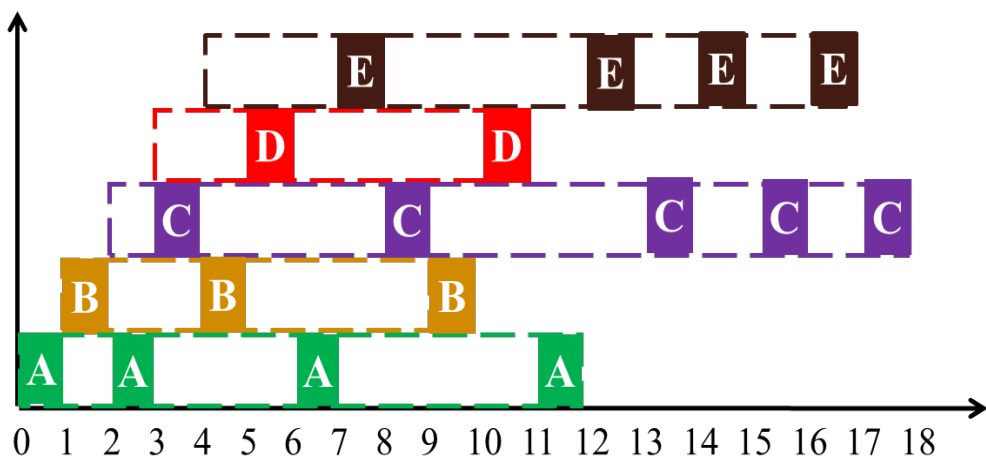
调度算法

进程调度算法周转时间比较

周转时间
响应时间
优先权



进程名	A	B	C	D	E	平均
到达时间	0	1	2	3	4	
服务时间	4	3	5	2	4	
周转时间 (Q=1)	12	9	16	8	13	11.6
周转时间 (Q=5)	4	6	10	11	14	9





处理机调度



■ 周转时间
■ 响应时间
■ 优先权

4. 最高优先权优先调度算法 (FPF)

作业调度

从后备队列中选择一个或多个**优先权最高**的作业，将它们调入内存、分配资源、创建进程，放入就绪队列。

进程调度

从就绪队列中选择一个**优先权最高**的进程，为之分配处理机。

多数操作系统中采用的进程调度算法

非抢占

进程一直运行到完成；或发生某事件而阻塞，系统将处理机分配给另一个优先级最高的进程。
批处理系统、要求不高的实时系统

抢占式

进程执行中，调度程序可随时停止其执行，将处理机分配给新出现的优先级更高的进程。
实时系统、交互系统



周转时间
响应时间
优先级

4. 最高优先级优先调度算法 (FPF)

静态优先级

创建进程时确定，且进程的整个运行期间保持不变。

进程类型：系统进程 > 用户进程

资源需求：需求少的 > 需求多的

优点：简单，不需要维护优先级。
缺点：高者恒高，不断地有高者转为就绪的话，低者将会“饥饿”。

例：某多道程序系统采用**抢占式静态优先级进程调度算法**。某段时间内有A、B、C三个进程，优先级 $C > B > A$ 。就绪时刻、计算与I/O所需时间如下表所示：

进程	进程就绪时刻	计算时间	I/O操作时间	计算时间
A	0ms	15ms	10ms	5ms
B	5ms	25ms	15ms	10ms
C	10ms	5ms	20ms	10ms

若采用多道方式运行，给出这三个进程运行完成总共所需的时间（忽略进行系统调度所需时间），采用多道方式运行比采用单道方式运行节省多少时间。



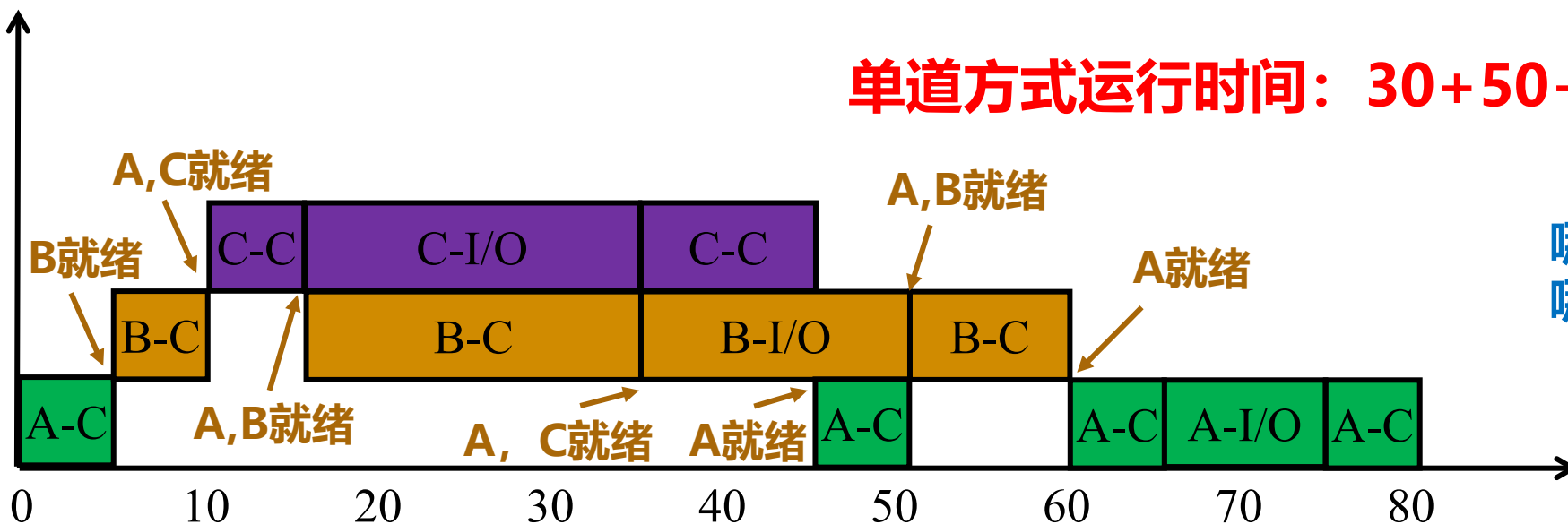
处理机调度



周转时间
响应时间
优先级

4. 最高优先级优先调度算法 (FPF)

进程	进程就绪时刻	计算时间	I/O操作时间	计算时间
A	0ms	15ms	10ms	5ms
B	5ms	25ms	15ms	10ms
C	10ms	5ms	20ms	10ms



哪些是抢占调度?
哪些是非抢占调度?



处理机调度



周转时间
响应时间
优先级



4. 最高优先权优先调度算法 (FPF)

动态优先级

系统为刚生成的进程赋初始优先级，之后根据进程的行为动态调整优先权的值

防止高者恒高，低者“饥饿”。

如何调整？

正在使用CPU的进程优先级下降；很久未得到CPU的进程优先级升高；I/O密集型进程拥有较高的优先级

间隔太短，开销太大；间隔太长，不能及时反映变化
一般选择调度、时钟中断、陷入等时机调整

何时调整？



主要内容

- 2.1 进程及其调度控制与死锁
- 2.2 UNIX的进程
- 2.3 中断的基本概念及UNIX中断处理
- 2.4 UNIX时钟中断与系统调用
- 2.5 UNIX的进程调度状态
- 2.6 进程通讯



进程对资源使用情况的建模

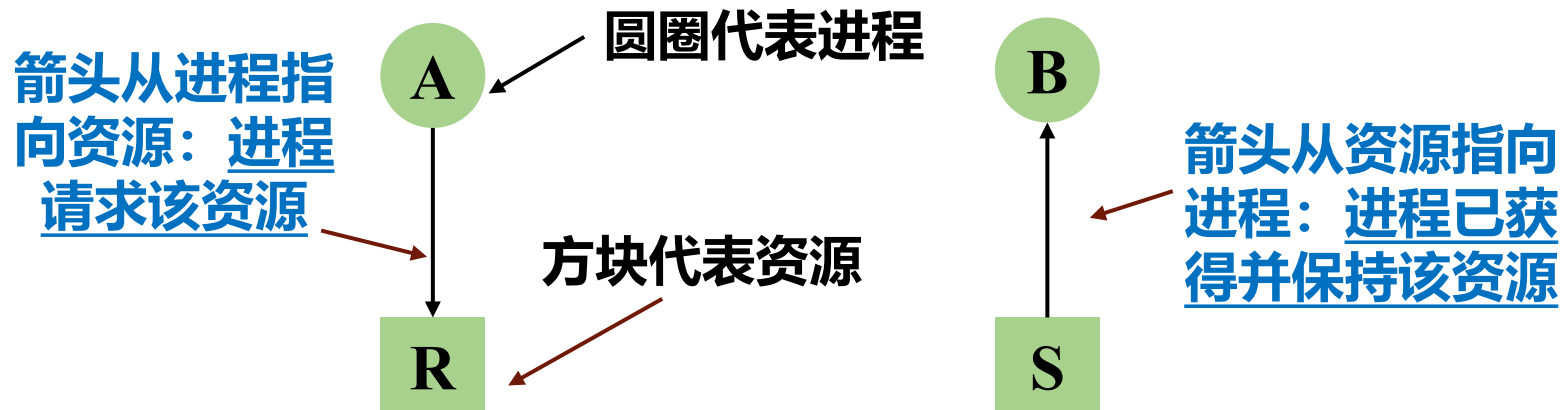


资源共享



各种程序活动的相互依赖与制约

进程在执行过程中会向系统申请使用各种软硬件资源。





进程对资源使用情况的建模



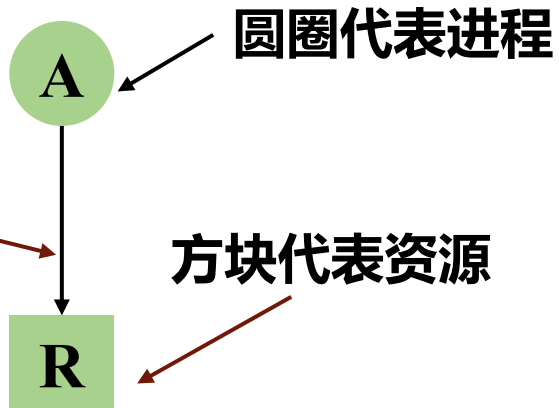
资源共享



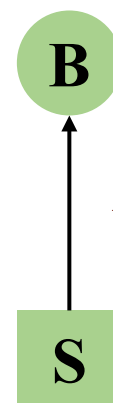
各种程序活动的相互依赖与制约

进程在执行过程中会向系统申请使用各种软硬件资源。

箭头从进程指向资源：进程请求该资源

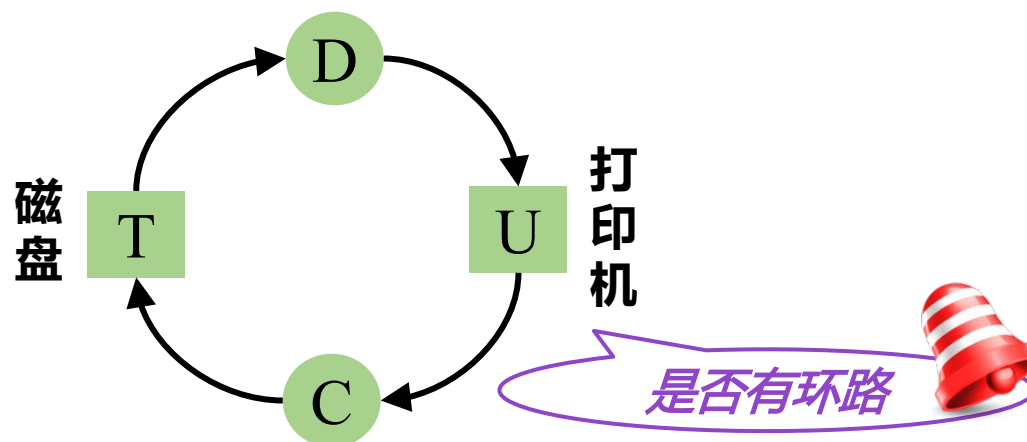


箭头从资源指向进程：进程已获得并保持该资源



死锁：一个进程集合中的每个进程都等待着永远都不会发生的事件而形成一种**永久阻塞状态**

死锁可以通过有向图来建模





死锁产生的原因



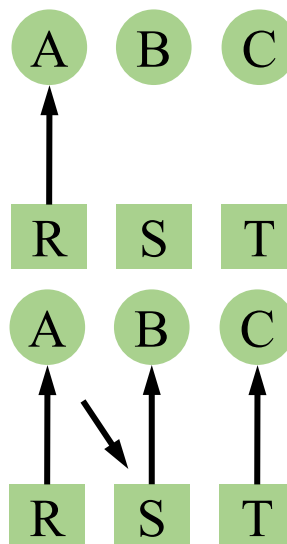
产生的原因

1. 资源竞争引起资源死锁
2. 进程推进顺序不当造成死锁

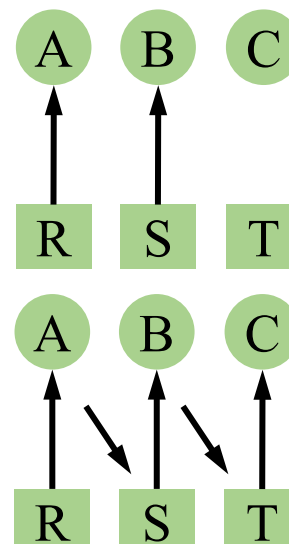
• 可剥夺资源 和 不可剥夺资源
对不可剥夺资源的竞争使用可能导致死锁

1. A Request R
2. B Request S
3. C Request T
4. A Request S
5. B Request T
6. C Request R

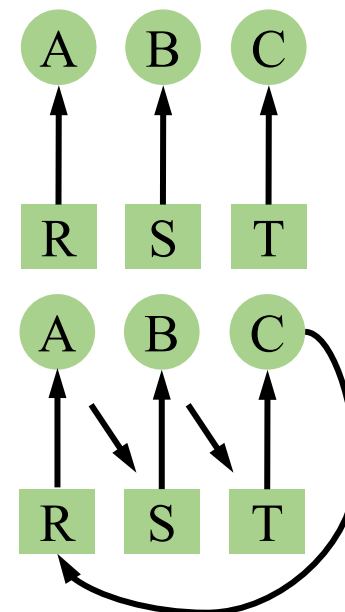
Process A
Request R
Request S
Release R
Release S



Process B
Request S
Request T
Release S
Release T



Process C
Request T
Request R
Release T
Release R





死锁产生的原因



产生的原因

1. 资源竞争引起资源死锁
2. 进程推进顺序不当造成死锁

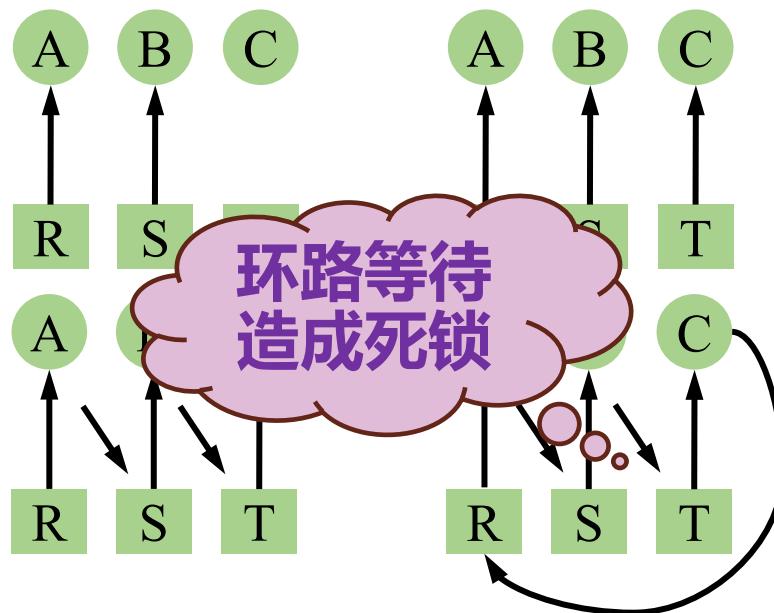
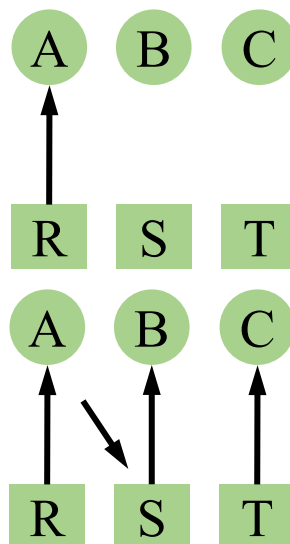
• 可剥夺资源 和 不可剥夺资源
对不可剥夺资源的竞争使用可能导致死锁

1. A Request R
2. B Request S
3. C Request T
4. A Request S
5. B Request T
6. C Request R

Process A
Request R
Request S
Release R
Release S

Process B
Request S
Request T
Release S
Release T

Process C
Request T
Request R
Release T
Release R





死锁产生的原因



产生的原因

1. **资源竞争**引起资源死锁
2. 进程**推进顺序**不当造成死锁

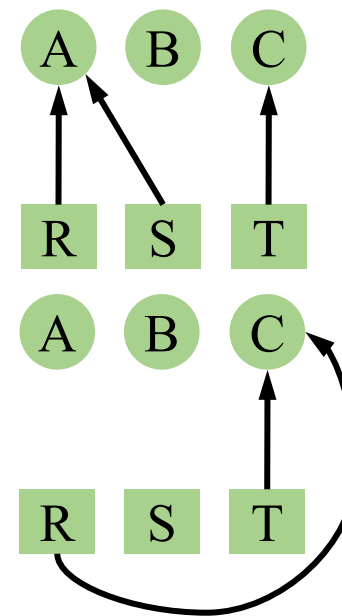
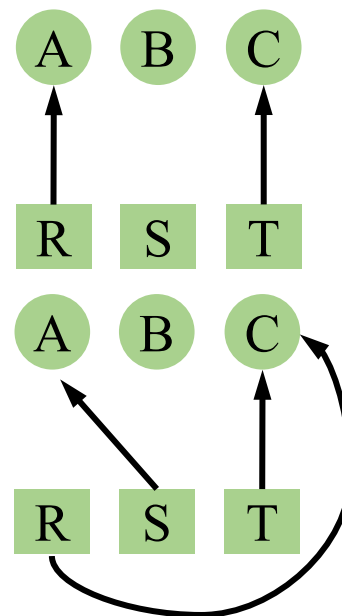
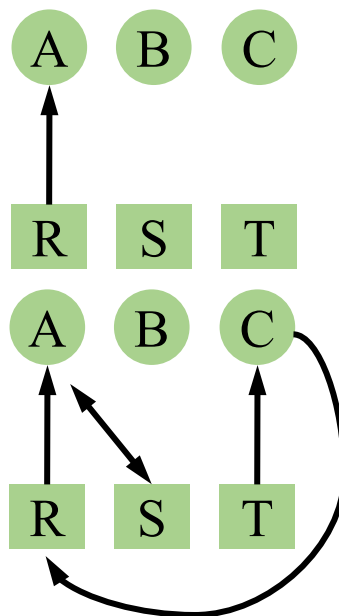
• 可剥夺资源 和 不可剥夺资源
对不可剥夺资源的竞争使用可能导致死锁

1. A Request R
2. C Request T
3. A Request S
4. C Request R
5. A Release R
6. A Release S

Process A
Request R
Request S
Release R
Release S

Process B
Request S
Request T
Release S
Release T

Process C
Request T
Request R
Release T
Release R





死锁产生的原因



产生的原因

1. 资源竞争引起资源死锁
2. 进程推进顺序不当造成死锁

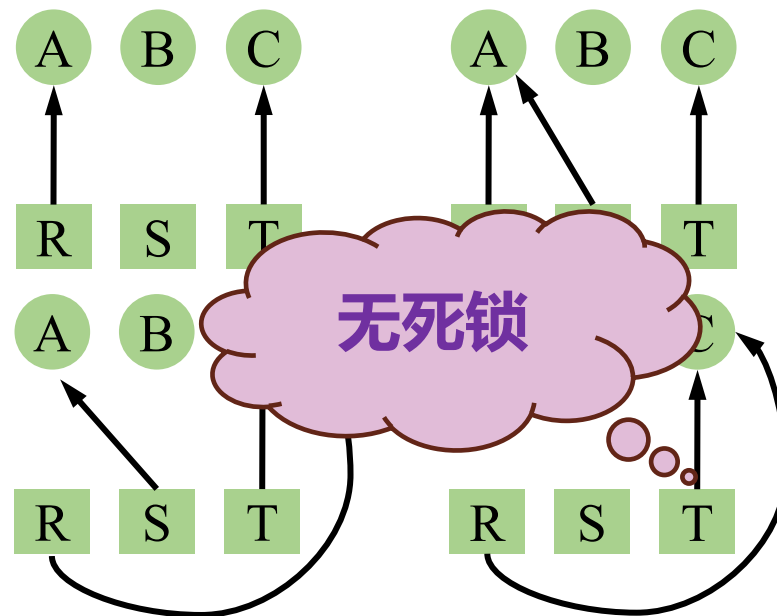
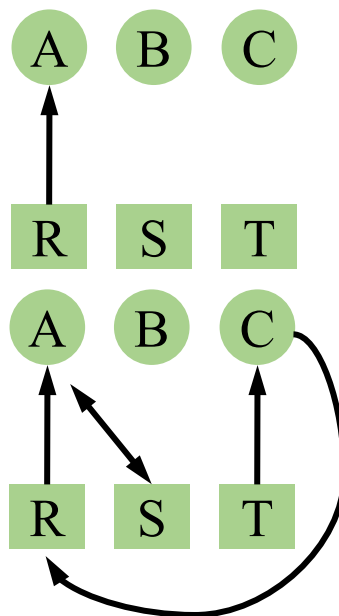
• 可剥夺资源 和 不可剥夺资源
对不可剥夺资源的竞争使用可能导致死锁

1. A Request R
2. C Request T
3. A Request S
4. C Request R
5. A Release R
6. A Release S

Process A
Request R
Request S
Release R
Release S

Process B
Request S
Request T
Release S
Release T

Process C
Request T
Request R
Release T
Release R





死锁产生的必要条件



1. **互斥条件**：即在一段时间内**某资源只能由一个进程占用**。如果此时还有其他进程请求使用该资源，则请求者只能等待，直至占用该资源的进程用毕释放。
2. **请求和保持条件**：进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源又已被其他进程占用，此时请求进程阻塞，但又**对自己已获得的其他资源保持不放**。
3. **不剥夺条件**：指进程**已获得的资源**，在未使用完之前，不能被剥夺，只能在使用完后由自己释放。
4. **环路等待条件**：指在发生死锁时，**必然存在由两个或多个进程构成的环形链**，环形链中的每一个进程都在等待下一个进程释放其占用的资源。



应对死锁的基本方法



1. 预防死锁

通过施加限制条件，破坏死锁的四个必要条件

2. 避免死锁

在资源的动态分配过程中，用某种方法去防止系统进入**不安全状态**

3. 检测死锁

允许系统在运行过程中发生死锁，但可以及时检测到死锁的发生，并精确地确定与死锁有关的进程和资源

4. 解除死锁

将进程从死锁状态中解脱出来

事先预防

配套使用
事后措施



1. 预防死锁：通过施加限制条件，破坏死锁的四个必要条件

条件	方法
互斥条件	设备的固有特性决定： <i>不能破坏</i>
请求保持条件	静态分配资源： <i>资源浪费严重，进程延迟执行</i> 一次性请求所有需要的资源，只有当 所有资源都可用时才分配给进程



1. 预防死锁：通过施加限制条件，破坏死锁的四个必要条件

条件	方法
互斥条件	设备的固有特性决定： <i>不能破坏</i>
请求保持条件	静态分配资源： <i>资源浪费严重，进程延迟执行</i>
不剥夺条件	剥夺资源： <i>实现代价比较大，可能失效</i> <i>一般适用于易保存和恢复状态的资源，如CPU。</i>

允许系统抢占部分或全部资源，确保高优先级的进程能够及时获得所需的资源，这可能涉及到挂起低优先级的进程，以便为高优先级进程腾出资源。



1. 预防死锁：通过施加限制条件，破坏死锁的四个必要条件

条件	方法
互斥条件	设备的固有特性决定： <i>不能破坏</i>
请求保持条件	静态分配资源： <i>资源浪费严重，进程延迟执行</i>
不剥夺条件	剥夺资源： <i>实现代价比较大，可能失效</i>
环路等待条件	为资源顺序编码， 并按需申请分配： <i>顺序难确定，新设备加入困难 增加程序员负担</i>

首先给系统中的资源编号，规定每个进程必须按编号递增的顺序请求资源，同类资源（即编号相同的资源）一次申请完。

一个进程只有已占有小编号的资源时，才有资格申请更大编号的资源。按此规则，已持有大编号资源的进程不可能逆向地回来申请小编号的资源，从而就不会产生循环等待的现象。



1. 预防死锁：通过施加限制条件，破坏死锁的四个必要条件

条件	方法
互斥条件	设备的固有特性决定： <i>不能破坏</i>
请求保持条件	静态分配资源： <i>资源浪费严重，进程延迟执行</i>
不剥夺条件	剥夺资源： <i>实现代价比较大，可能失效</i>
环路等待条件	为资源顺序编码， 并按需申请分配： <i>顺序难确定，新设备加入困难 增加程序员负担</i>

假设系统中有10个资源，编号为1, 2, ... 10 P1拥有资源1, 3; P1拥有资源2, 4; P1拥有资源5, 7;
在任何一个时刻，总有一个进程拥有的资源编号是最大的，这个进程申请之后的资源必然畅通无阻。因此，不可能出现所有进程都阻塞的死锁现象)



2. 避免死锁：在资源的动态分配过程中，防止系统进入**不安全状态**

安全状态

系统能按某个进程顺序（安全序列）来为每个进程分配所需资源，直至对资源的最大需求，使每个进程都可顺利地完成。

系统中进程对某一资源的需求如下：

进程	最大需求	已分配	还需
P1	10	5	5
P2	4	2	2
P3	9	2	7
可用：3			

存在**安全序列** <P2, P1, P3>

允许进程**动态地申请资源**，但系统在**进行资源分配之前**，首先计算此次资源分配的安全性，若此次分配**不会导致系统进入不安全状态**，则将**资源分配给进程**，否则进程等待。



应对死锁的基本方法



2. 避免死锁：在资源的动态分配过程中，防止系统进入**不安全状态**

最大需求矩阵Max

	A	B	C	D
P1	0	6	5	6
P2	1	9	4	2
P3	1	3	5	6
P4	1	7	5	0

需求矩阵Need

	A	B	C	D
P1	0	6	4	2
P2	1	7	2	0
P3	0	0	0	2
P4	0	7	5	0

已分配的资源矩阵Allocation

	A	B	C	D
P1	0	0	1	4
P2	0	2	2	2
P3	1	3	5	4
P4	1	0	0	0

目前系统中可利用的资源向量

Available			
A	B	C	D
2	7	3	0

请求资源向量Request

1. *if* Request[i] > Need[i] **error**;
2. *if* Request[i] > Available **block()**;
3. **//尝试资源分配**
Available -= Request[i];
Allocation[i] += Request[i];
Need[i] -= Request[i];
4. *if* (**safety()**) 分配
else {不分配, 恢复所有数据结构的原始值。}



应对死锁的基本方法



Max				
	A	B	C	D
P1	0	6	5	6
P2	1	9	4	2
P3	1	3	5	6
P4	1	7	5	0

Allocation				
	A	B	C	D
P1	0	0	1	4
P2	0	2	2	2
P3	1	3	5	4
P4	1	0	0	0

Need				
	A	B	C	D
P1	0	6	4	2
P2	1	7	2	0
P3	0	0	0	2
P4	0	7	5	0

Available				
	A	B	C	D
	2	7	3	0

进程P2提出需求:

1 2 1 0

首先, 尝试分配:

Allocation				
	A	B	C	D
P1	0	0	1	4
P2	<u>1</u>	<u>4</u>	<u>3</u>	<u>2</u>
P3	1	3	5	4
P4	1	0	0	0

Need				
	A	B	C	D
P1	0	6	4	2
P2	<u>0</u>	<u>5</u>	<u>1</u>	<u>0</u>
P3	0	0	0	2
P4	0	7	5	0

Available				
	A	B	C	D
	<u>1</u>	<u>5</u>	<u>2</u>	<u>0</u>

再判断分配后的状态是否安全



应对死锁的基本方法



	Max			
	A	B	C	D
P1	0	6	5	6
P2	1	9	4	2
P3	1	3	5	6
P4	1	7	5	0

	Allocation			
	A	B	C	D
P1	0	0	1	4
P2	1	4	3	2
P3	1	3	5	4
P4	1	0	0	0

	Need			
	A	B	C	D
P1	0	6	4	2
P2	0	5	1	0
P3	0	0	0	2
P4	0	7	5	0

Available			
A	B	C	D
1	5	2	0



1 4 3 2



Available			
A	B	C	D
2	9	5	2



1 3 5 4



Available			
A	B	C	D
3	12	10	6

Finish

P1 FALSE
P2 FALSE
P3 FALSE
P4 FALSE

从P1开始，依次查找Available可以满足哪一个进程执行结束

P2可执行结束，结束后释放全部资源

Finish

P1 FALSE
P2 TRUE1
P3 FALSE
P4 FALSE

继续查找Available可以满足哪一个进程执行结束

P3可执行结束，结束后释放全部资源



应对死锁的基本方法



Max				
	A	B	C	D
P1	0	6	5	6
P2	1	9	4	2
P3	1	3	5	6
P4	1	7	5	0

Allocation				
	A	B	C	D
P1	0	0	1	4
P2	1	4	3	2
P3	1	3	5	4
P4	1	0	0	0

Need				
	A	B	C	D
P1	0	6	4	2
P2	0	5	1	0
P3	0	0	0	2
P4	0	7	5	0

Available			
A	B	C	D
3	12	10	6

+

1 0 0 0

||

Available			
A	B	C	D
4	12	10	6

+

0 0 1 4

||

Available			
A	B	C	D
4	12	11	10

Finish
P1 FALSE
P2 TRUE1
P3 TRUE2
P4 FALSE

继续查找Available可以满足哪一个进程执行结束

P4可执行结束，结束后释放全部资源

Finish
P1 FALSE
P2 TRUE1
P3 TRUE2
P4 TRUE3

循环查找Available可以满足哪一个进程执行结束

P1可执行结束，结束后释放全部资源



应对死锁的基本方法



Max				
	A	B	C	D
P1	0	6	5	6
P2	1	9	4	2
P3	1	3	5	6
P4	1	7	5	0

Allocation				
	A	B	C	D
P1	0	0	1	4
P2	<u>1</u>	<u>4</u>	<u>3</u>	<u>2</u>
P3	1	3	5	4
P4	1	0	0	0

Need				
	A	B	C	D
P1	0	6	4	2
P2	<u>0</u>	<u>5</u>	<u>1</u>	<u>0</u>
P3	0	0	0	2
P4	0	7	5	0

Available			
A	B	C	D
<u>1</u>	<u>5</u>	<u>2</u>	<u>0</u>

Finish

P1 TRUE4
P2 TRUE1
P3 TRUE2
P4 TRUE3

尝试资源分配之后，可以找到一个安全序列 $\langle P2, P3, P4, P1 \rangle$

所以，此次资源分配请求可以满足

如果找不到这样的安全序列，则此次资源分配请求不能满足，各矩阵恢复原值。



3. 检测死锁：能够检测到系统是否进入死锁状态

在资源分配图中检测到环路。环路中的进程为发生死锁的进程

4. 解除死锁：将进程从死锁状态中解脱出来

- 终止所有死锁进程
- 逐个终止死锁进程，直至死锁不再存在（应从终止代价最小的进程开始）。
- 强迫死锁进程释放资源（以损失最小为依据）



5. 无作为：鸵鸟策略





本节小结



- 1 经典的调度算法
- 2 死锁的概念及银行家算法

阅读教材：41页 ~ 47页；62页 ~ 71页



E02：并发进程（基本概念、处理机调度与死锁）